ALGORITMOS E ESTRUTURAS DE DADOS · BCC 2026.2

Rede de distribuição

Como o algoritmo funciona, quanto custa e como as peças se juntam na saída

Entregável 1 · histórico de períodos

Entregável 2 · fechamento e proximidade

Entregável 3 · rede de rotas

Autores: Vinicius Lima, Laércio Neto e Zaque Mateus

VISÃO GERAL

O problema, em uma frase

Uma rede de depósitos evolui por períodos: a cada lote de operações, depósitos entram, saem ou mudam de lucro, e precisamos responder consultas com eficiência (até 5000 depósitos vivos e $3 \cdot 10^5$ operações).

Entregável 1

Histórico

Lucro de cada período, melhor intervalo (MELHOR) e pilha de prosperidade (PROSPERIDADE).

Hash · pilha amortizada · programação dinâmica

Entregável 2

Fechamento e proximidade

Fechar depósitos abaixo da tolerância (REMOVER) e achar o par mais próximo (PAR).

Heap com alteração de chave · divisão e conquista

Entregável 3

Rede de rotas

Menor custo de um depósito aos demais (DIST) num grafo que cresce e encolhe.

Grafo esparso · Dijkstra com o mesmo heap

COMO TUDO SE JUNTA

O programa, de ponta a ponta

Os três arquivos seguem o mesmo esqueleto; cada entregável acrescenta uma peça.



- `main` consome a entrada e chama `executar_lote` uma vez por período.
- Cada comando (INSERIR, LUCRO, REMOVER, MELHOR…) é um método pequeno que mexe nas estruturas e, se for consulta, devolve **uma linha**.
- As linhas vão para uma lista e saem juntas no fim, na ordem em que foram executadas.
- Consultas veem o estado do ponto do lote em que estão; L_k só é fixado **depois** do lote.

e1.py

```
def executar_lote(leitor, registro, pilha, historico, saida):
    q = leitor.inteiro()
    for _ in range(q):
        comando = leitor.palavra()
        if comando == "INSERIR":
            tratar_inserir(leitor, registro)
        elif comando == "LUCRO":
            tratar_lucro(leitor, registro)
        elif comando == "MELHOR":
            saida.append(historico.descricao())
        elif comando == "PROSPERIDADE":
            saida.append(pilha.descricao())

def fechar_periodo(k, registro, pilha, historico):
    lk = registro.total
    pilha.registrar(k, lk)
    historico.registrar(k, lk)
```

ENTREGÁVEL 1

Registro por id e lucro do período

1. Ids chegam a 10^9 e há até 5000 vivos: vetor por id é inviável. Usamos um `dict` (tabela hash).
2. Guardamos também `total`, a soma de todos os lucros.
3. INSERIR soma o lucro; LUCRO soma só a **diferença** (`novo - antigo`).
4. Fim do lote: L_k `= total`, sem percorrer depósitos.

Complexidade: INSERIR e LUCRO em **O(1) esperado**; obter L_k em **O(1)**. Espaço O(n).

e1.py

```python
def inserir(self, ident, lucro):
    self.lucro[ident] = lucro
    self.total += lucro

def atualizar_lucro(self, ident, novo):
    self.total += novo - self.lucro[ident]
    self.lucro[ident] = novo
```

ENTREGÁVEL 1

Pilha de prosperidade

1. $L_k > 0$: empilha (k, L_k).
2. $L_k = 0$: nada muda.
3. $L_k < 0$: o prejuízo consome o topo; se sobra, desempilha e continua no próximo.
4. A soma dos resíduos fica num acumulador, então PROSPERIDADE só lê base, topo e soma.

Amortizado O(1): cada período empilha no máximo 1 elemento, e cada elemento sai uma única vez. Em P períodos são ≤ P desempilhamentos no total (pior período isolado: O(h)).

e1.py

```python
def registrar(self, k, lk):
    if lk > 0:
        self.itens.append((k, lk))
        self.soma += lk
    elif lk < 0:
        d = -lk                          # prejuízo ainda a compensar
        while d > 0 and self.itens:
            j, r = self.itens[-1]
            if r > d:
                # o topo absorve todo o prejuízo e sobra resíduo;
                # mantém o índice j do período em que o ganho ocorreu
                self.itens[-1] = (j, r - d)
                self.soma -= d
                d = 0
            else:
                # o topo é consumido por inteiro (r <= d): sai da pilha
                self.itens.pop()
                self.soma -= r
                d -= r
```

ENTREGÁVEL 1

MELHOR: Kadane online

1. Recorrência: `suf(k) = L_k + max(0, suf(k-1))`: estende o intervalo corrente ou recomeça em k.
2. Estende com soma anterior `≥ 0`: no empate fica o menor `i`.
3. O melhor global só troca com soma **estritamente maior**: no empate fica o menor `j`.
4. O intervalo nunca é vazio, mesmo com todos os L negativos.

Complexidade: registrar um período e responder MELHOR em **O(1)**, com **5 inteiros** de estado (espaço O(1)); não guardamos o histórico.

e1.py

```python
def registrar(self, k, lk):
    if self.vazio or self.suf_soma < 0:
        self.suf_soma = lk
        self.suf_i = k
    else:
        self.suf_soma += lk
    if self.vazio or self.suf_soma > self.melhor_soma:
        self.melhor_soma = self.suf_soma
        self.melhor_i = self.suf_i
        self.melhor_j = k
    self.vazio = False
```

ENTREGÁVEL 1

Como o Entregável 1 gera a saída

Comando	O que faz	Custo
INSERIR	cria no `dict`, soma ao total, descarta as rotas	O(1) esperado
LUCRO	atualiza o total pela diferença	O(1) esperado
MELHOR	imprime `i j S` do estado de Kadane	O(1)
PROSPERIDADE	imprime `h b s i r` (ou 0)	O(1)
Fechar período	L_k = total → pilha e Kadane	O(1) amortizado

Juntando tudo: cada período fecha com L_k = `total`, que alimenta **duas** estruturas independentes: a pilha (PROSPERIDADE) e o Kadane (MELHOR). O programa inteiro roda em tempo **linear no número de comandos** (O(1) por comando); as rotas são só puladas na entrada.

ENTREGÁVEL 2

Heap indexado: alterar o lucro

1. Heap de mínimo ordenado pelo lucro: o menor lucro fica sempre na raiz.
2. Um `dict` `pos` (id → posição) acha qualquer depósito sem varrer o heap.
3. LUCRO troca a chave: se **diminuiu**, sobe (`_subir`); se **aumentou**, desce.
4. Cada elemento movido tem sua posição atualizada em `pos`.

Complexidade: achar o id **O(1) esperado** + reposicionar **O(log n)**. Por isso não usamos `heapq`, que não altera a chave de um elemento qualquer.

e2.py

```python
def _subir(self, i):
    ids, chaves, pos = self.ids, self.chaves, self.pos
    ident, chave = ids[i], chaves[i]
    while i > 0:
        pai = (i - 1) >> 1
        if chave < chaves[pai]:
            ids[i] = ids[pai]
            chaves[i] = chaves[pai]
            pos[ids[i]] = i
            i = pai
        else:
            break
    ids[i] = ident
    chaves[i] = chave
    pos[ident] = i

def alterar_chave(self, ident, nova):
    i = self.pos[ident]
    antiga = self.chaves[i]
    self.chaves[i] = nova
    if nova < antiga:
        self._subir(i)
    elif nova > antiga:
        self._descer(i)
```

ENTREGÁVEL 2

REMOVER sem varrer todos

1. A raiz do heap é o menor lucro.
2. Enquanto `lucro < T` (estrito), extrai a raiz, apaga o depósito e conta.
3. Quando a raiz é `≥ T`, ninguém mais está abaixo: para na hora.
4. Imprime a quantidade removida.

Complexidade: O(k log n) para k removidos: só toca em quem será fechado, nunca nos n depósitos todos. Depósito abaixo de T *continua vivo* até o REMOVER.

e2.py

```python
def remover_abaixo(self, tolerancia):
    fila = self.fila
    removidos = 0
    while len(fila) > 0 and fila.minimo()[1] < tolerancia:
        ident, _ = fila.extrair_minimo()
        del self.coord[ident]
        removidos += 1
    return removidos
```

ENTREGÁVEL 2

PAR: divisão e conquista (1/2)

1. Ordena os pontos por x uma vez e divide ao meio.
2. Resolve cada metade recursivamente (δ_E, δ_D) e fica com δ = o menor.
3. Devolve também os pontos ordenados por y (como um merge sort) para o nível de cima.
4. Tudo em **inteiros**: comparamos distâncias **ao quadrado**.

Por quê? Comparar todos os pares é $O(n^2)$. Dividindo, o trabalho de cada nível é linear e há log n níveis.

e2.py (_par_recursivo, divisão)

```
    meio = (lo + hi) // 2
    x_meio = pts[meio][0]                       # reta divisória
    esq, y_esq = _par_recursivo(pts, lo, meio)
    dir_, y_dir = _par_recursivo(pts, meio, hi)
    melhor = esq if esq < dir_ else dir_        # delta = min(delta_E, delta_D)
    por_y = sorted(y_esq + y_dir, key=itemgetter(1))
```

ENTREGÁVEL 2

PAR: a faixa central (2/2)

1. Um par melhor que δ pode ter um ponto em cada metade; ambos estão a menos de δ da reta divisória: a **faixa**.
2. Percorre a faixa ordenada por y e só compara com os próximos enquanto `dy < δ`.
3. Cada ponto vê no máximo **7** vizinhos (casa dos pombos num retângulo 2δ×δ).
4. Distância final = teto da raiz: `isqrt(d²)` (+1 se não for exata).

Complexidade: T(n) = 2T(n/2) + O(n) = **O(n log n)** por consulta. Sem matriz nem heap de distâncias; guardamos só o **último resultado** (cache).

e2.py (_par_recursivo, faixa central)

```
    d2_min = melhor[0]
    delta = math.isqrt(d2_min) + 1              # >= delta real: faixa um pouco larga
    esquerda, direita = x_meio - delta, x_meio + delta
    faixa = [p for p in por_y if esquerda < p[0] < direita]
    tam = len(faixa)
    for i in range(tam):
        xi, yi, idi = faixa[i]
        for j in range(i + 1, tam):
            xj, yj, idj = faixa[j]
            dy = yj - yi
            if dy * dy >= d2_min:
                break       # a faixa está ordenada por y: os próximos são piores
            d2 = (xi - xj) ** 2 + dy * dy
            if d2 < d2_min:
                d2_min = d2
                melhor = (d2, min(idi, idj), max(idi, idj))
    # Por que o laço interno é O(1): em cada metade os pontos distam >= delta
    # entre si; um retângulo 2*delta x delta da faixa comporta no máximo 8
    # desses pontos (princípio da casa dos pombos em 8 quadrados de
    # delta/2), logo cada ponto vê no máximo 7 vizinhos antes do break.
    return melhor, por_y
```

ENTREGÁVEL 2

Como o Entregável 2 gera a saída

Comando	O que faz	Custo
INSERIR	entra no heap e nas coordenadas; invalida o cache	O(log n)
LUCRO	reposiciona no heap (não invalida o cache)	O(log n)
REMOVER	extrai da raiz enquanto lucro < T; imprime a quantidade	O(k log n)
PAR	divisão e conquista; imprime `a b d`	O(n log n) · cache O(1)

Juntando tudo: o heap serve de registro de depósitos *e* de índice de lucros; o PAR só depende das coordenadas dos vivos. O cache é descartado só quando o conjunto de vivos muda (INSERIR ou REMOVER que fecha ≥ 1), então LUCRO não força recalcular.

ENTREGÁVEL 3

Grafo com listas de entrada e de saída

1. `saida[u]` = {v: custo}: rotas que saem de u.
2. `entrada[v]` = {u}: quem tem rota para v.
3. INSERIR adiciona a rota nas duas listas.
4. Remover u: percorre `saida[u]` e `entrada[u]`, apagando u dos vizinhos, e depois apaga u.

Por que duas listas? Só com a de saída, achar quem chega em u custaria O(n + m). Com as duas, remover custa **O(grau de u)**. Matriz de adjacência gastaria $O(n^2)$ de espaço; a lista usa **O(n + m)**.

e3.py

```
def adicionar_rota(self, u, v, custo):
    self.saida[u][v] = custo
    self.entrada[v].add(u)

def remover_vertice(self, u):
    for v in self.saida[u]:
        self.entrada[v].discard(u)
    for w in self.entrada[u]:
        del self.saida[w][u]
    del self.saida[u]
    del self.entrada[u]
```

ENTREGÁVEL 3

DIST: Dijkstra com o heap do Entregável 2

1. Começa com `dist[s] = 0` e s no heap.
2. Extrai o vértice de menor distância provisória: ela vira definitiva.
3. **Relaxa** suas rotas: se `dist[u] + c < dist[v]`, atualiza v (decrease-key; insere se for novo).
4. Repete até o heap esvaziar; quem não foi alcançado vale −1.

Por que funciona: com custos ≥ 1, nenhum caminho por vértice ainda aberto pode ser mais barato que o menor provisório.
Complexidade: n extrações + até m decrease-keys, cada um O(log n): **O((n + m) log n)**.

e3.py

```python
def menores_custos(self, origem):
    dist = {origem: 0}
    fila = FilaPrioridadeMin()
    fila.inserir(origem, 0)
    saida = self.saida
    while len(fila) > 0:
        u, du = fila.extrair_minimo()
        for v, c in saida[u].items():
            nd = du + c
            if v not in dist:
                dist[v] = nd
                fila.inserir(v, nd)
            elif nd < dist[v]:
                dist[v] = nd
                fila.alterar_chave(v, nd)
    return dist
```

ENTREGÁVEL 3

Como o Entregável 3 gera a saída

1. O Dijkstra devolve a distância dos alcançáveis.
2. Os ids vivos são ordenados (não são contíguos, vão até 10^9).
3. Para cada id ≠ s imprime a distância ou `-1`.
4. Se s é o único vivo, a linha sai vazia.

Juntando tudo: REMOVER usa o heap de lucros (E2) **e** apaga o depósito do grafo; DIST usa outra instância da **mesma** classe de heap. INSERIR: O(log n + r); REMOVER: O(k log n + graus); DIST: O((n+m) log n + n log n).

e3.py

```
def dist(self, leitor):
    s = leitor.inteiro()
    dist = self.registro.grafo.menores_custos(s)
    vivos = sorted(self.registro.grafo.saida)
    return " ".join(str(dist.get(v, -1)) for v in vivos if v != s)
```

RESUMO

Complexidades de tudo

Operação	Estrutura / algoritmo	Custo
Registro por id	tabela hash (`dict`)	O(1) esperado
L_k, PROSPERIDADE, MELHOR	acumulador, pilha, Kadane	O(1) (pilha: amortizado)
LUCRO / INSERIR	heap indexado	O(log n)
REMOVER	heap: extrai enquanto lucro < T	O(k log n + graus)
PAR	divisão e conquista + cache	O(n log n)
DIST	Dijkstra com o mesmo heap	O((n + m) log n)

n = depósitos vivos · **m** = rotas vivas · **P** = períodos · **h** = altura da pilha · **k** = removidos. Nenhuma operação varre todos os depósitos sem necessidade.

VERIFICAÇÃO

Como testamos

- Referência **ingênua** (força bruta) das 7 operações, conferida com o exemplo do enunciado.
- **400 entradas aleatórias** por entregável, comparando saída linha a linha: **0 falhas em 1200 execuções**.
- Valores pequenos de propósito, para forçar empates, prejuízo que zera a pilha, inalcançáveis e remoção com rotas.
- Pior caso (5000 depósitos, 100 mil períodos, 500 PAR, 200 DIST): e1 ≈ 0,5 s; e2 e e3 de 10 a 27 s.

Limite honesto: o limite de tempo da correção é desconhecido; e2 e e3 são os que mereceriam otimização adicional se ele for apertado.